AI Engineering and Claude Architect · Part II, Retrieval

RAG, Lesson 2

Ingestion and Chunking

The refund rule was in the index. It was split across two chunks, with "Orders above $5,000 require" in one and "approval from a regional manager" in the next. Search found the first half, and the assistant told a customer that they needed nothing more.

Module	AI05 of 13
Lesson	2 of 7
Prerequisites	AI05 L1, AI04 L1, HLD M21B (chunking basics)
Exam relevance	Out of scope as a topic
Reading time	~35 minutes

Contents

1. Ingestion and Chunking

Lesson 2 of 7 · RAG

Ingestion and Chunking

Key idea

A chunk is the unit your system can find and the unit your model reads, so a fact split across two chunks is, for practical purposes, lost. Structure-aware chunking keeps facts whole far more cheaply than overlap does. **Most RAG quality is decided at ingestion**, in parsing, chunking and metadata, long before any model runs. Treat ingestion as a real data pipeline, with versions, deletes and permissions.

> *Before: HLD M21B gives the basics: 300–800 token chunks, overlap, and natural boundaries. Teaches: measuring chunking quality, structure-aware splitting, parent-child retrieval, parsing, metadata, deletes and permissions. Used by: AI05 L3, AI05 L7, AI08 L5.*

The hook: a rule cut in half

Consider what fixed-size chunking does to a policy document. It cuts wherever the character count says, including in the middle of a sentence.

Here's a measurement, on a short invented refund policy with 9 specific facts. A fact counts as **broken** if no single chunk contains it whole. Real output from `code/ai05-rag/.../ChunkLab.java`:

```
document=804 chars, facts=9
fixed 100, no overlap      chunks= 9 stored= 804 chars broken facts=3 of 9 cut mid-sentence=8
fixed 200, no overlap      chunks= 5 stored= 804 chars broken facts=3 of 9 cut mid-sentence=4
fixed 200, 20% overlap     chunks= 5 stored= 964 chars broken facts=1 of 9 cut mid-sentence=4
fixed 200, 50% overlap     chunks= 8 stored=1504 chars broken facts=0 of 9 cut mid-sentence=7
fixed 400, no overlap      chunks= 3 stored= 804 chars broken facts=1 of 9 cut mid-sentence=2
structured, max 200        chunks= 7 stored= 921 chars broken facts=0 of 9 cut mid-sentence=0
structured, max 400        chunks= 4 stored= 836 chars broken facts=0 of 9 cut mid-sentence=0
```

> *Key idea*
>
> **Structure-aware chunking kept every fact whole, with about 836 stored characters. Fixed-size chunking needed 50% overlap and 1,504 characters to match that, and it still cut seven chunks mid-sentence.**

This is one tiny, clean document with invented facts, and sizes in characters for simplicity. Real results depend on your documents. **But the method is general: define what must stay together, and measure it.**

Intuition: cutting a recipe book

Suppose you cut a recipe book into index cards.

Method	Result
Every 200 characters	Half an ingredient list on one card, and the oven temperature on another
With overlap	More cards, repeated text, and the same awkward cuts
One card per recipe section, labeled with the recipe's name	Each card makes sense when you pick it up

The last row has the important part: **each card carries its heading**, so it stands on its own.

The ingestion pipeline

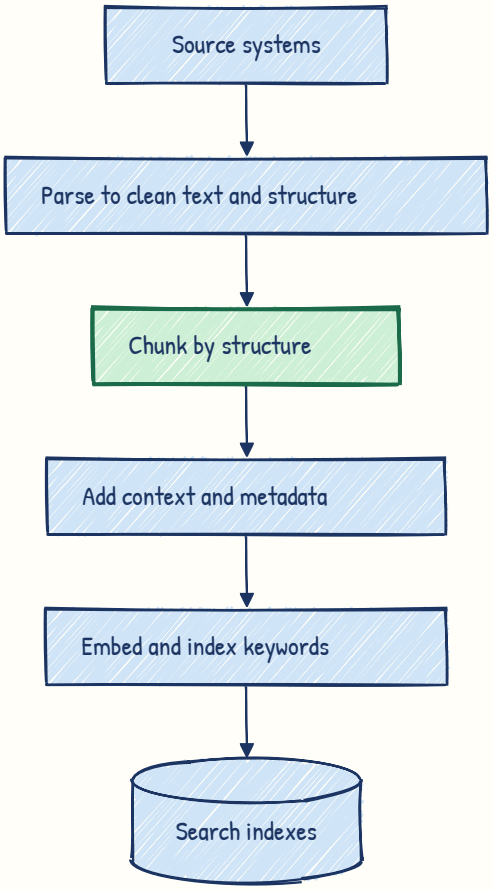


Figure 1. Ingestion stages. Each one can lose information, and quality problems found later usually started here.

Parsing: where quality really begins

If parsing mangles a document, nothing downstream can recover it.

Source	What goes wrong	Mitigation
PDF	Multi-column text interleaved, headers repeated on each page	Layout-aware parser. Sample and read the output
Tables	Rows lose their column meaning	Keep tables whole, as Markdown or as rows with headers repeated
Scanned PDFs	No text layer	OCR. Note that Claude's citations need extractable text
HTML	Menus and footers indexed as content	Extract the main content
Slides	Fragments with no sentences	Add slide titles to each piece
Wikis	Stale or duplicate pages	Filter by status and date

The cheapest quality check in RAG: read 50 of your parsed chunks. Many teams never do.

Chunking strategies

Strategy	How	Good for	Weakness
Fixed size	Every N tokens	Uniform text with no structure	Cuts anywhere
Fixed with overlap	Repeats the edges	A cheap improvement	Storage, and duplicates in results
Recursive	Split on headings, then paragraphs, then sentences	Most documents	Needs readable structure
Structure-aware	Uses the format's own units: sections, functions, table rows	Policies, code, tables	Needs a parser per format
Semantic	Splits where embedding similarity drops	Long unstructured prose	Costs more, and results vary
Parent-child	Search small pieces, return the larger parent	Precise matching with enough context	More storage and logic
Late chunking or contextual models	Embed in document context	Chunks that depend on context	Model-dependent (AI04 L1)

Here's the structure-aware version from the lab. It keeps the heading path with each chunk, so it stands alone:

```
String heading = title + " > " + lines[0].replace("## ", "").trim();
String body = lines.length > 1 ? lines[1].trim() : "";
StringBuilder current = new StringBuilder(heading).append(":");
for (String sentence : body.split("(?<=\\.)\\s+")) {
    if (current.length() + sentence.length() + 1 > maxSize
            && current.length() > heading.length() + 1) {
        chunks.add(current.toString());
        current = new StringBuilder(heading).append(":");
    }
    current.append(' ').append(sentence);
}
```

And real output:

```
Refund Policy > Refund timing: Card refunds take 5 to 10 business days to appear. Wallet refunds appear within 24 hours. Bank transfers take up to 14 days because of clearing rules.
```

The sentence splitter here is deliberately simple. It would stumble on "e.g." or "$5.00", so use a proper library for production.

Choosing a size

There's no universal best size, and HLD M21B's 300–800 tokens is a common range.

Too small	Too large
Loses context	Mixes topics, so embeddings blur
Precise match, poor answer	Finds the right area, with wasted tokens
Many more vectors to store	Fewer, cheaper vectors

Use your eval (AI05 L6) to choose. Sweep a few sizes, and compare recall and answer quality.

Parent-child retrieval

This separates **what you match on** from **what the model reads.**

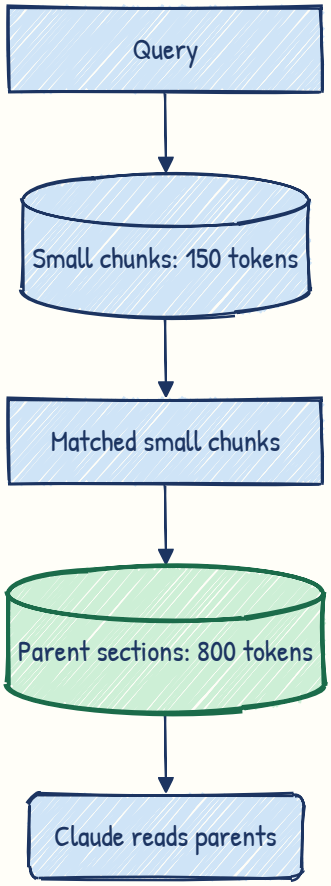


Figure 2. Parent-child retrieval. Small chunks give precise matches, and the model receives the larger section, so it has enough context to answer.

Deduplicate parents, since several small chunks may share one.

Metadata: what makes chunks usable

Field	Used for
`document_id`, `version`, `chunk_no`	Idempotent upserts, and updates
`source_url`, `title`, `heading_path`, `page`	Citations (AI05 L4)
`published_at`, `effective_from`	Freshness, and time-based filtering (TS 5.6)
`tenant_id`, `allowed_groups`	**Permissions inside the search**
`language`, `doc_type`, `region`	Filters and routing
`content_hash`	Skip unchanged chunks (AI04 L1)
`embedding_model`	The version guard (AI04 L1)

Duplicate documents are a quiet problem. Five copies of an old policy can outrank the one current version, since retrieval sees five good matches. Deduplicate, and prefer the authoritative source.

Keeping the index correct over time

The hard part isn't the first load. It's staying correct on day 400.

Event	Handling
Document edited	Re-chunk it, then upsert changed chunks, using hashes to skip the rest
Chunk count shrinks	**Delete leftovers**, or old sections stay searchable
Document deleted	Tombstone at once, and remove from all indexes
Permissions changed	Update metadata quickly, and re-check at read time
Policy superseded	Retire it, or use effective dates
Chunking strategy changes	Rebuild, as with a model change (AI04 L1)

The leftovers case is a classic bug: an edit shrinks a document from 12 chunks to 9, an upsert overwrites 9, and chunks 10–12 remain searchable forever. **Write by document version, then atomically publish that version**, as the production design below does.

How the field and Anthropic approach it

Problem	Approach	Source
Chunks lose context	Add document context to each chunk	Anthropic's Contextual Retrieval, AI05 L3
Boundaries affect retrieval	That post lists chunk size, boundaries and overlap as considerations	Same
Citations need chunks	Citations auto-chunk plain text and PDFs into sentences, and custom content documents let you define your own	Citations docs
Chunks embedded without context	Contextual embedding models	Voyage guide

A practical tip from the Citations docs: if you want Claude to cite specific sentences within your RAG chunks, put each chunk in a plain text document. For no further chunking, use a custom content document.

The production architecture

This is a recommended reference design, not a specific company's system.

Baseline: a one-time script. It loads everything once. Nothing handles edits, deletes or failures.

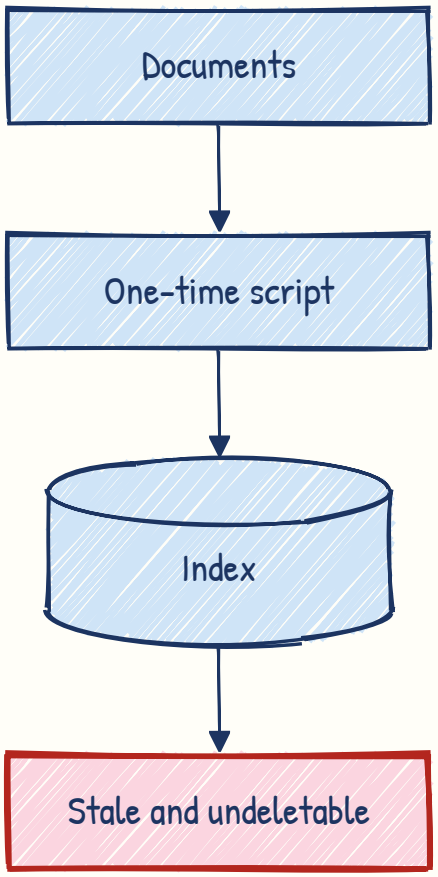


Figure 3. Baseline. A one-off script gives a good demo, then the index slowly drifts away from reality.

Improved: event-driven updates. Changes flow through continuously. New problem: **partial updates are visible, and shrinking documents leave orphan chunks behind.**

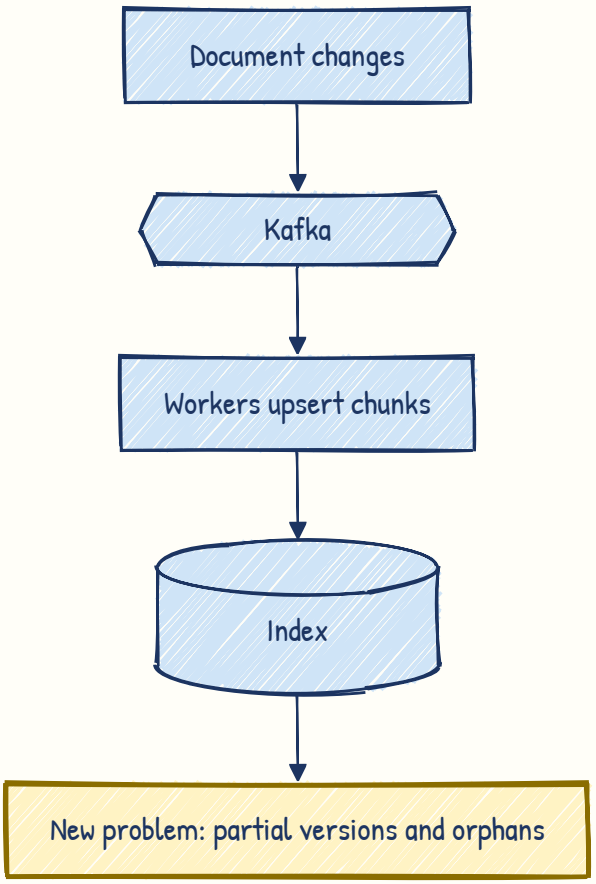


Figure 4. Improved. The index keeps up with changes, but a half-written document can appear in search, and leftovers remain.

Production: versioned publishing.

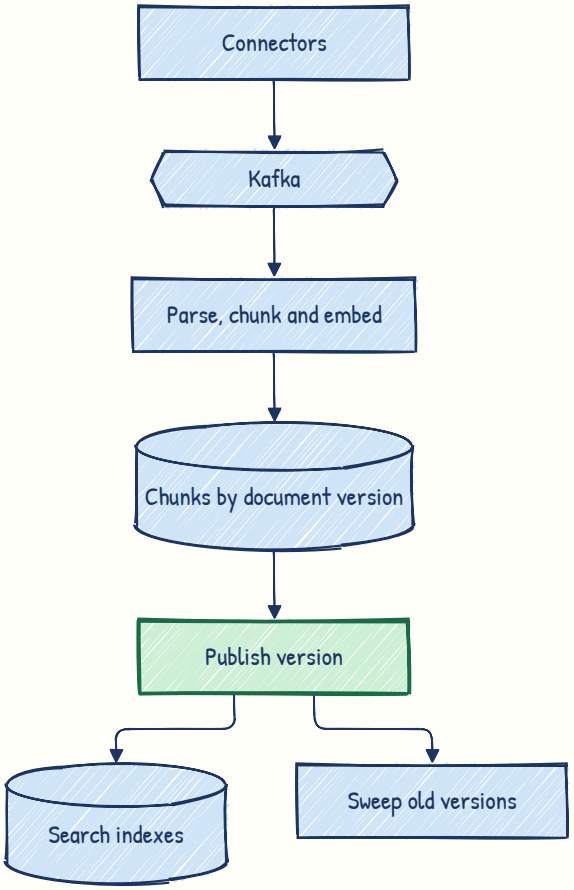


Figure 5. Production. Each document version is written completely before it's published. Search only reads published versions, so edits appear atomically, and old chunks can be swept safely.

Component	Why it exists	How it communicates
Connectors	Pull from wikis, drives and tickets	Webhooks or polling, emitting events
Workers	Do the slow processing, safely retried	Kafka consumers keyed by document ID, so edits stay in order
Chunk store	Durable record by version	Rebuilds indexes (AI04 L5)
Publish step	Atomic visibility	Updates the active version pointer
Sweeper	Removes superseded chunks	Scheduled job

Failures

Failure	Containment
A parser update breaks a format	Sample parsed output in CI, by document type
Poison document	Retry limit, then a dead-letter queue, with an alert
Embedding provider outage	Queue backs up, and search stays on the last published version
Connector misses a deletion	Periodic full reconciliation against the source
Permission sync lag	Re-check at read time for sensitive sources

Trade-offs

Decision	Gain	Cost
Overlap	Fewer split facts	Storage, and duplicate results
Structure-aware	Whole facts, meaningful chunks	A parser per format
Heading path in each chunk	Chunks stand alone, and it improves search	Tokens
Parent-child	Precise match, plus context	Complexity
Versioned publishing	Atomic updates	More storage and logic

When simple is fine: a small corpus of uniform articles. Split by heading, and move on.

Common mistakes

1. **Never reading your parsed chunks.** Do it first.
2. **Trusting overlap to fix bad boundaries.** Fix the boundaries.
3. **Losing table structure.** A row without its headers is meaningless.
4. **Forgetting orphan chunks.** Shrinking documents leave leftovers.
5. **Omitting permissions from metadata.** Search can't filter what it can't see.
6. **Ignoring duplicates.** Stale copies can outrank the current policy.
7. **Only testing the first load.** Test edits and deletes.

What to say in an interview

"Most RAG quality is decided at ingestion. I'd chunk by structure, keep heading paths with each chunk, and measure with a fact-integrity test, since structure-aware chunks kept every fact whole in my test at half the storage of heavy overlap. I'd publish document versions atomically, because shrinking documents otherwise leave orphan chunks searchable forever."

Certification lens

How this lesson maps to CCAR-F

Not tested as a topic. One idea connects: chunking oversized documents on failure appears in **TS 4.5**, and preserving metadata supports **TS 5.6**.

These are original practice questions for understanding, not exam simulations.

Q Q1 · Troubleshooting · Medium. After a policy edit, users still see the old rule in search results. The new text is also there. What's the likely bug, and the fix?

A Orphan chunks: the new version has fewer chunks, so upserts overwrote some, and left the rest. Write complete document versions, publish atomically, and sweep old versions. A reconciliation job catches anything already orphaned.

Q Q2 · Calculation · Easy. A 1,000-token document is split into 200-token chunks with 50 tokens of overlap. How many chunks, and how many tokens are stored?

A The step is 150. Chunks start at 0, 150, ... 750, and that one reaches the end at 950, so one more starts at 900 to reach 1,000. That's **7 chunks**: six of 200 tokens and a last one of 100, storing **1,300 tokens**. Overlap adds 30% here.

Q Q3 · Best decision · Hard. Retrieval misses many questions on a product manual full of tables. A colleague suggests a better embedding model. What do you check first?

A How the tables were parsed. If rows lost their column headers, no embedding model can recover their meaning. Read the parsed chunks, keep each table whole or repeat headers on rows, then re-evaluate. Only then compare embedding models.

Revision - Lesson 2

→ A fact split across chunks is effectively lost.

→ Measure chunking with a fact-integrity test on your own documents.

→ In the lab, structure-aware chunking kept all facts whole, using 836 characters. Fixed-size needed 50% overlap and 1,504.

→ Keep the heading path with each chunk, so that it stands alone.

→ Parsing comes first. Read 50 parsed chunks.

→ Parent-child: match small chunks, and let the model read larger parents.

→ Metadata: version, source, dates, permissions, hash and embedding model.

→ Duplicates can outrank the current version.

→ Shrinking documents leave orphan chunks, so publish versions atomically.

→ Reconcile against the source periodically.

Your notes